

DATA 266 - Homework 6 Findings

Self-Supervised and Contrastive Representation Learning on STL-10

Personal Parameters

Parameter	Value
SID4	9486
SEED	9486
SLICE	486
HP_ID	0
CLS_A	6
CLS_B	0

SEED 9486 is used for Python, NumPy, and PyTorch, for choosing the 500 labeled images and the 20,000 SimCLR images, and for DataLoader shuffling and worker seeds. SLICE, CLS_A (horse), and CLS_B (airplane) decide the query images for the nearest-neighbor part. HP_ID is only reported.

Introduction

For this assignment I wanted to see how much the way an encoder is trained matters when labels are scarce. I trained the same ResNet-18 encoder (torchvision, weights=None, 512-d output after global average pooling) on STL-10 in three ways: supervised training on only 500 labeled images, rotation prediction as a self-supervised pretext task, and SimCLR-style contrastive learning. For the two self-supervised encoders, I measure quality with the standard linear evaluation protocol: freeze the encoder, train only a linear layer on the same 500 labels, and report test accuracy. I started from the professor's Demo_6_Self_Supervised_Learning.ipynb and kept its RotSSL and TwoViews datasets, nt_xent loss, SimCLR augmentations, and nearest-neighbor code, moving them from CIFAR-10 and a 3-layer CNN to STL-10 at 96x96 and ResNet-18. Every number here comes from one run of HW6_STL10_SSL.ipynb on a Colab Tesla T4 (about 62 minutes, fp32, cuDNN deterministic mode on).

Main results

Model	Encoder training	Labeled training	Test accuracy
A: Supervised	end to end with the labels	15 epochs, all parameters	34.80%
B: Rotation SSL + linear	rotation prediction, 15 epochs, 100,000 unlabeled images	20 epochs, linear head only	42.58%
C: SimCLR + linear	NT-Xent ($\tau = 0.2$), 20 epochs, 20,000 unlabeled images	20 epochs, linear head only	50.45%

All three use the same 500 labeled training images (50 per class, the same index list in every part) and are tested on all 8,000 STL-10 test images after the last epoch.

Setup

Item	Value
Data	STL-10: train 5,000, test 8,000, unlabeled 100,000, all 96x96 RGB, kept at 96x96 and normalized with mean and std from the labeled train split

Item	Value
Encoder	resnet18(weights=None) with fc replaced by Identity, 11,176,512 parameters, same for all three
Part A	encoder + Linear(512, 10), Adam lr 1e-3, batch 32, 15 epochs, random crop + flip
Part B pretraining	encoder + Linear(512, 4), one random 0/90/180/270 rotation per image (demo's RotSSL), Adam lr 3e-4, batch 256, 15 epochs
Part C pretraining	encoder + MLP head 512 to 512 to 128, two views per image with the demo's RandomResizedCrop (0.6-1.0), HorizontalFlip, ColorJitter (0.4, 0.4, 0.4, 0.1), RandomGrayscale (0.2), NT-Xent with cosine similarity and tau = 0.2, Adam lr 3e-4, batch 256, 20 epochs
Linear evaluation	head removed, encoder frozen (requires_grad=False and kept in eval mode), new Linear(512, 10) trained with Adam lr 1e-3, batch 32, 20 epochs

Checks that passed in the reported run: the labeled subset has exactly 500 images with the same fingerprint in Parts A, B, and C; the encoders use weights=None; rotation labels stay in 0-3 and all four appear about 375,000 times; all 120 encoder tensors (weights and BatchNorm buffers) are unchanged after both linear evaluations; the similarity used in NT-Xent matches F.cosine_similarity; and the same three query images are used for every encoder.

How Each Method Works

Rotation SSL. Each unlabeled image is rotated by $k \times 90$ degrees with k drawn uniformly from $\{0, 1, 2, 3\}$, and the encoder plus a Linear(512, 4) head is trained with 4-way cross entropy to predict k . Chance is 25%. The label comes for free, but to solve it the network has to pick up on things like where the sky and ground are and which way objects are facing.

SimCLR. Each image in a batch of $B = 256$ is augmented twice, giving $2B = 512$ views. The projection head maps each 512-d embedding to 128-d, the outputs are L2-normalized, and the 512×512 matrix of dot products (which equals the cosine similarity) is divided by $\tau = 0.2$. For each view, the other view of the same image is the positive and the remaining 510 views are negatives, and NT-Xent is the cross entropy of picking out the positive. A small τ sharpens the softmax, so hard negatives get more weight. I checked that the matrix the loss uses matches F.cosine_similarity, with a maximum difference of $6.6e-7$.

Linear evaluation. After pretraining, the rotation head or projection head is dropped. The encoder's parameters get requires_grad=False, and the encoder is kept in eval mode so the BatchNorm running statistics do not change either. Only a Linear(512, 10) layer (5,130 parameters) is trained. Comparing all 120 tensors in the encoder's state_dict before and after confirmed that none of them changed, so the accuracy reflects only what the frozen features already encode.

Training Behavior

Part	Start	End
A supervised: train / test accuracy	20.40% / 18.36%	60.80% / 34.80%
B rotation pretraining: loss / rotation accuracy	1.1396 / 45.23%	0.7292 / 62.58%
B linear evaluation: train / test accuracy	15.00% / 15.01%	45.00% / 42.58%
C SimCLR pretraining: NT-Xent loss	4.2217	2.1757
C linear evaluation: train / test accuracy	20.60% / 36.75%	67.60% / 50.45%

The train/test gap after the last epoch says a lot about each model. The supervised model ended at 60.80% train vs 34.80% test, a 26-point gap. From epoch 3 on, its test accuracy jumped around between 30.58% and 40.74%, and after the first two epochs its test loss bounced between 1.67 and 2.30, which is classic overfitting with 11.18M trainable parameters and 500 examples. The SimCLR probe ended at 67.60% train vs 50.45% test. Its test accuracy was already 44.96% after 2 epochs and stayed around 50 to 51% from epoch 8 on, so the linear layer found most of what it could early. The rotation probe is the opposite case: train and test stayed close (45.00% vs 42.58%) and were both still rising at epoch 20, which looks like underfitting, so more epochs or a higher learning rate for the probe would probably help. Both pretext tasks were also still improving when they stopped: rotation accuracy reached 62.58% (chance is 25%) with the loss still falling, and the NT-Xent loss dropped from 4.2217 to 2.1757 and had not flattened out.

Why SimCLR Did Best

SimCLR's objective lines up best with classification. To make the NT-Xent loss small, the encoder has to map two random crops, flips, and color or grayscale versions of the same image close together, so it becomes invariant to exactly the things that do not change the class (position, scale, color). At the same time, the 510 negatives in each batch push different images apart, which spreads the embeddings out over the sphere. Both properties make the classes easier to separate with a single linear layer, which is why the frozen SimCLR features reached 50.45%.

Rotation prediction beat training from scratch by about 8 points (42.58% vs 34.80%), so pretraining on 100,000 unlabeled images did give the encoder useful low and mid-level features. But the task asks the network to keep orientation information, not to become invariant to anything that changes within a class. An upright car and an upright ship share the same orientation cues (sky at the top, a long horizontal shape), so nothing in the rotation loss forces the encoder to separate them, and the nearest neighbors below show exactly that.

The supervised model had the right objective but too little data. With about 22,000 parameters per labeled image and only crop and flip augmentation, it fit the training set without learning features that generalize, which shows up as the 26-point train/test gap.

Part D: Nearest Neighbors

Queries were chosen by a fixed rule from SLICE, CLS_A, and CLS_B: test image 498 (horse), 491 (airplane), and 486 (ship). Each encoder's test embeddings were L2-normalized, so the dot product is the cosine similarity, and the query itself was excluded from its top 5.

Query	Supervised	Rotation SSL	SimCLR
498, horse	3/5 (monkey, bird)	3/5 (dog, deer)	5/5
491, airplane	5/5	5/5	5/5
486, ship	2/5 (car, car, truck)	0/5 (car, car, truck, truck, truck)	3/5 (car, airplane)
Total same-class	10/15	8/15	13/15

Airplanes against a plain sky were easy for all three. For the horse, SimCLR returned five horses in different colors, poses, and settings, while rotation SSL's mistakes were a dog and a deer, both four-legged animals standing sideways, which fits with it having learned shape and orientation. The supervised encoder's mistakes (a monkey in a tree and an owl) share little with the query beyond brownish texture. The ship query showed the biggest difference: rotation SSL returned only cars and trucks, all long

vehicles lying horizontally in the frame, while SimCLR found three ships, including an almost identical white ship in front of a city skyline.

The cosine values also say something about the embedding spaces. Rotation SSL's top-5 similarities were 0.98 to 1.00 for every query, even for wrong neighbors, which suggests its test embeddings sit in a narrow cone where everything looks similar, so small differences decide the ranking. SimCLR's were about 0.88 to 0.92 for the horse and ship queries. That fits with the contrastive denominator pushing different images apart. Since retrieval only uses the ranking, this does not directly affect accuracy, but it helps explain why the SimCLR neighbors were more consistent.

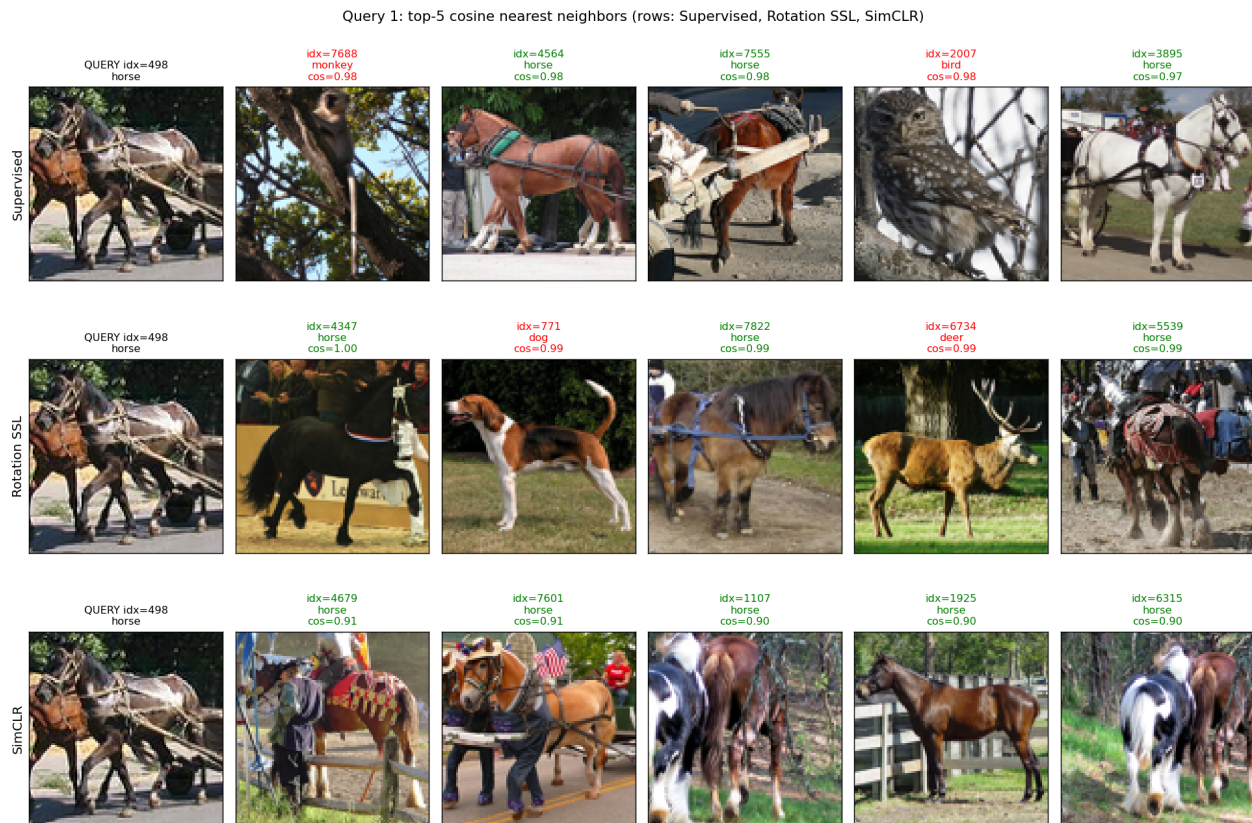


Figure 1. Query 498 (horse). Rows: Supervised, Rotation SSL, SimCLR. Green titles match the query's class, red titles do not.

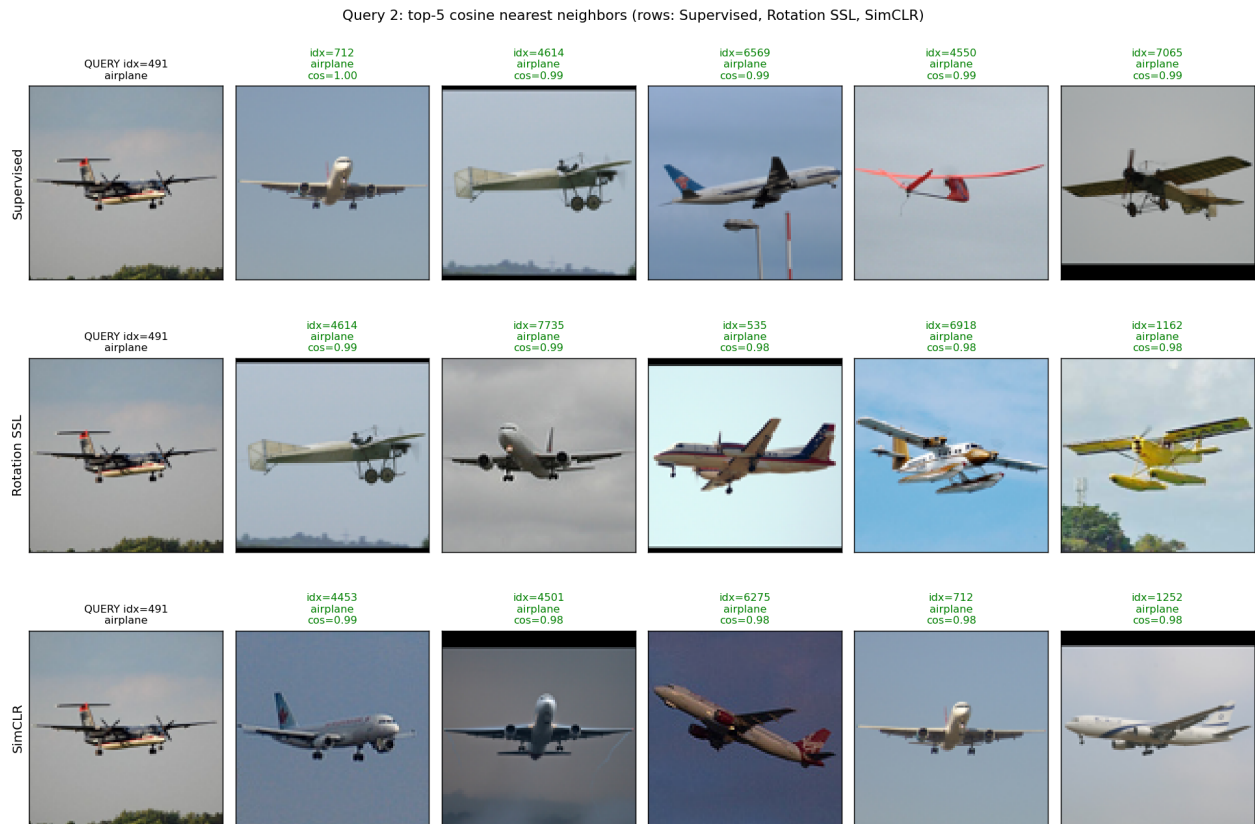


Figure 2. Query 491 (airplane). All three encoders return five airplanes.

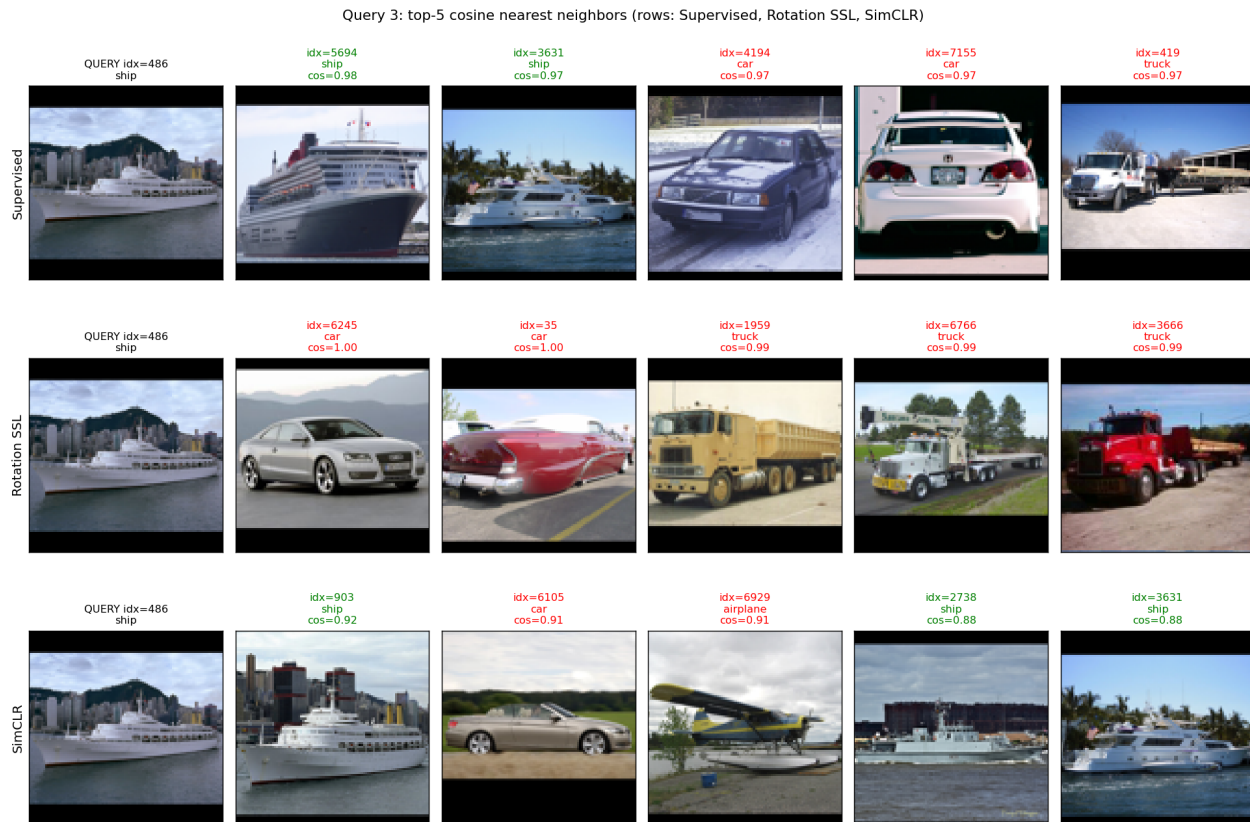


Figure 3. Query 486 (ship). Rotation SSL returns only cars and trucks; SimCLR finds three ships.

What Could Improve Each Model

Supervised: stronger augmentation, a learning rate schedule to calm the jumpy test accuracy, and a validation split from the other 4,500 labeled images for early stopping. Fine-tuning one of the self-supervised encoders instead of starting from random weights would probably help the most.

Rotation SSL: more pretraining epochs, since the rotation loss was still dropping, and more linear evaluation epochs, since that curve had not leveled off. Showing all four rotations of each image in one batch is another option.

SimCLR: more epochs, since the loss was still falling at epoch 20, all 100,000 unlabeled images instead of 20,000, a larger batch for more negatives, and trying stronger augmentations or a different temperature. I kept the demo's augmentations and $\tau = 0.2$ as required, so none of that was tested here.

Limitations

This is one run with one seed, so the gaps between the models could change with another seed, especially for the supervised model. The nearest-neighbor counts come from only three queries, so they are examples, not a measured retrieval score. All training was short (15 to 20 epochs) compared with the hundreds of epochs self-supervised methods normally use, and no hyperparameters were tuned because a single run took about an hour on a T4. Rotation SSL used five times as many unlabeled images as SimCLR (as the assignment specifies), and SimCLR still came out ahead.

Final findings

With the same ResNet-18 and the same 500 labels, how the encoder was trained made a big difference: SimCLR reached 50.45%, rotation SSL 42.58%, and supervised training from scratch 34.80%. Both self-supervised methods beat the supervised baseline using only unlabeled data before seeing any labels, and the contrastive method beat the rotation method even though it saw a fifth of the unlabeled images. The nearest-neighbor figures tell the same story: SimCLR groups images by what the object is, rotation SSL groups them more by shape and layout, and the supervised encoder falls in between.